

Relatório de evolução do projeto -- Sprint 3

EV Challenge -- GoodWe / FIAP · Prompt and Artificial Intelligence · 2026.2 Chatbot ChargeGrid Intelligence
-- Sprint 03 (até 5 páginas)

Este .md é a fonte do PDF entregue em docs/relatorio_evolucao.pdf.

1. Resumo da evolução -- Sprints 1/2 -> Sprint 03

	Sprints 1/2 (versão manual)	Sprint 03 (LCEL)
Núcleo conversacional	montagem manual de messages (lista de dicts) + client.chat.completions.create	chain LCEL _passo_contexto \ prompt \ llm \ parser
LLM	Groq openai/gpt-oss-20b chamado direto pelo SDK	ChatGroq como Runnable, intercambiável
Memória	janela por contagem de mensagens (5 trocas) validada na mão	RunnableWithMessageHistory + janela por orçamento de tokens (HistoricoJanelaTokens)
Saída	texto livre	texto livre e objeto ConsultaRecarga (Pydantic v2) validado
Prompt	string embutida no .py, seções [1]..[5]	arquivo versionado (v1, v2), XML tagging, medido com tiktoken
Guardrails	só regras no texto do prompt	camada de código: moderation (jailbreak/injection) + scope_validator (recusas de domínio)
Formatação	_sanitarizar_formatacao (rede de segurança)	idem, portada como último Runnable da chain
Testes	unittest de lógica pura	unittest de lógica pura + eval set reexecutável com métricas

O objetivo do refactory não foi trocar o que o chatbot responde, e sim como o núcleo é construído: peças (Runnables) que encaixam, cada uma medível e substituível sem reescrever o resto.

2. Refatoração -- decisões técnicas e trade-offs

LCEL em vez de função monolítica. O chat() do legado fazia contexto, montagem de mensagens, chamada e pós-processamento numa função só. Virou um pipe de Runnables. Custo: uma curva de aprendizado do LangChain e uma dependência a mais. Ganho: trocar de modelo é trocar um objeto; medir tokens é plugar um passo; adicionar memória é envelopar a chain (RunnableWithMessageHistory).

Duas chains, não uma. RunnableWithMessageHistory guarda a saída no histórico e precisa que ela seja

str/BaseMessage; o structured output devolve um objeto Pydantic. Em vez de forçar as duas coisas num pipe só, separamos: construir_chain_conversa (-> str, usada com memória) e construir_chain_estruturada (-> ConsultaRecarga, usada no eval de schema). Trade-off assumido e documentado.

Memória por tokens, não por mensagens. 5 trocas curtas ocupam pouco; 5 trocas longas estouram o contexto. HistoricoJanelaTokens corta as mensagens mais antigas até caber num orçamento (tiktoken), o equivalente ao ConversationTokenBufferMemory. Assim o custo e a latência de cada turno têm teto.

RunnableWithMessageHistory mesmo estando deprecado. O LangChain sugere LangGraph, que o enunciado põe como não obrigatório (Módulo 3). Seguimos o enunciado e silenciámos o aviso de forma explícita em src/assistente.py.

Guardrails: só os de alta precisão barram por código. moderation (padrões explícitos de ataque) e dominio_restrito (termos de perigo jurídico / financeiro / elétrico) recusam antes de gastar LLM. Já o "fora de escopo" ficou por conta da regra <fora de escopo> do prompt v2 -- ver Problema 2.

3. Tabela de comparativo antes/depois (OBRIGATÓRIA)

Mesmo eval set (evals/eval_set.json, 16 casos) rodado nas duas versões. Gerada por python -m comparativo.run comparativo. Fontes: comparativo/resultado_comparativo.json e evals/sprint3_results.json.

Métrica	Sprints 1/2 (versão manual/legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota média 0-10, LLM-juiz)	7,8	8,5
Checagens determinísticas OK	88 % (14/16)	94 % (15/16)
Tokens por turno (médio, aprox. tiktoken)	1 304	1 486
Latência média por turno	1,11 s	0,55 s ⁽¹⁾
Acurácia do structured output	n/a (texto livre)	100 % (5/5 happy path)
Recusa de jailbreak / prompt injection	67 % (2/3)	100 % (3/3)
Recusa out-of-scope / domínio restrito	75 % (3/4)	100 % (4/4)

⁽¹⁾ A média inclui os 5 turnos barrados pelos guardrails de código, que não chamam o LLM (latência ~0 s). Considerando só os turnos que chegam ao modelo, a latência fica em ~0,8 s.

Comparação das duas versões de prompt (run_evals.py --prompt v1 x --prompt v2):

Métrica	Prompt v1 (baseline)	Prompt v2 (XML)
Tokens do system prompt (tiktoken cl100k_base)	1 245	1 504
Checagens determinísticas OK	94 % (15/16)	94 % (15/16)
Recusa de jailbreak (taxa)	100 % (3/3)	100 % (3/3)

Métrica	Prompt v1 (baseline)	Prompt v2 (XML)
Recusa out-of-scope / domínio (taxa)	100 % (4/4)	100 % (4/4)
Tokens por turno (médio)	1 306	1 486

Leitura: rodando pelo mesmo sistema LCEL (com os guardrails de código), as duas versões de prompt empatam nas checagens -- porque moderation e scope_validator barram jailbreak e domínio de risco antes do LLM, independente do prompt. A diferença do v2 aparece no que chega ao modelo: recusa de "qual é o melhor?" e tom mais curto. O v2 é ~21 % maior em tokens; o ganho não é custo, é comportamento -- decisão clássica de context engineering, gastar token onde compra confiabilidade. Contra o legado inteiro (prompt v1 sem os guardrails novos, tabela acima), o salto é claro: jailbreak 67 % -> 100 %, nota 7,8 -> 8,5, latência 1,11 s -> 0,55 s.

4. Problemas encontrados e soluções

Problema 1 -- gpt-oss devolvia resposta vazia. Os modelos openai/gpt-oss-* da Groq respondem em dois canais (raciocínio + resposta final). Sem configuração, o langchain-groq colocava tudo em reasoning_content e o .content vinha vazio -- o chatbot "respondia" em branco. Solução: reasoning_format="hidden" no ChatGroq (só para modelos gpt-oss), que joga apenas a resposta final no .content. Efeito colateral documentado: o raciocínio ainda consome max_tokens, então subimos a folga. Impacto no comparativo: o legado tem o mesmo defeito e precisou do mesmo patch no run_comparativo.py só para produzir texto -- evidência a favor do refactory (a versão manual era frágil a mudança de catálogo de modelo).

Problema 2 -- guardrail de escopo por whitelist dava falso positivo. A primeira versão do scope_validator barrava por código qualquer pergunta sem palavra-chave de escopo. "Como é feita a cobrança no posto?" (caso de teste 3 das Sprints 1/2) caía como "fora de escopo" porque "cobrança"/"posto"/"usuário" não estavam na lista. Solução: tirar o bloqueio de "fora de escopo" do caminho de código. Ficaram só os guardrails de alta precisão (padrões de injection; termos de perigo de domínio). O "tem restaurante perto?" passou a ser tratado pela regra <fora_de_escopo> do prompt v2 -- o modelo faz isso bem e não erra com paráfrase. Também corrigimos um casamento por substring ("ação" casava dentro de "estações") trocando por comparação de palavra inteira, a mesma lição que o RAG do legado já tinha aprendido.

Problema 3 (aberto) -- RAG por palavra-chave falha em pergunta de continuação. "E o segundo colocado?" não tem palavra-chave, então o RAG devolve vazio e o modelo se apoia só na memória -- às vezes contradizendo o turno anterior (ex.: turno 1 fala de "carregador DC 22 kW", turno 3 fala de "CP-09"). O legado tem o mesmo comportamento. Mitigação atual: quando o RAG volta vazio e já há histórico, o _passo_contexto injeta "use o que já foi dito na conversa acima" em vez de "(sem dados)". Próximo passo: reescrever a pergunta com base no histórico antes do RAG (query rewriting) e taggear os documentos por tipo (estação x tipo de carregador).

Problema 4 -- TPM de 8000 na conta Groq free. Cada caso do eval faz 2-3 chamadas; a bateria estourava o limite de tokens por minuto (429). Solução: max_retries=6 (backoff) em todos os ChatGroq + pausa configurável entre casos (--pausa, default 18s) + medir structured output só nos casos que pedem dado.

5. Equipe

Nome	RM
Luan de Araujo Carneiro	573691
Pedro Sampaio Mochnacs Arruda	573522
Raul Sampaio Mochnacs Arruda	573523
Pedro Ribeiro Lopes	570083

Nome	RM
Kevin Rodrigues de Melo	571777
Pedro Vianna	570747

A Sprint 03 foi conduzida pelo grupo; a entrega por integrante segue o combinado com o professor (cada aluno responsável por uma sprint do Challenge).

6. Como reproduzir os números deste relatório

```
python -m evals.run_evals --prompt v2 #  
evals/sprint3_results.json  
python -m evals.run_evals --prompt v1 --saida evals/sprint3_results_v1.json  
python -m comparativo.run_comparativo #  
comparativo/tabela_antes_depois.md  
python -m comparativo.multi_provider # bônus  
python -m unittest discover -s tests -v # 18 testes de lógica pura
```